

Lecture 17

Selection Sort

- It sorts by repeatedly selecting the smallest or largest element from the unsorted portion and swapping it with the first unsorted element.
- Algorithm
 1. Find the smallest element and swap it with first element.
 2. Find the second smallest from remaining elements and swap it with second element.
 3. Find the third smallest element from remaining elements and swap it with the third element
 4. Continue this process until the all elements moved to correct position.

Advantages & Disadvantages of Selection Sort

Advantages

- It requires constant extra space to sort the array.
- It requires less number of swaps compared to other sorting algorithms.

Disadvantages

- It has time complexity $O(n^2)$ makes it slower compared to Quick Sort or Merge Sort.
- Selection sort is not stable sorting algorithm.

Merge Sort

- Merge Sort follows divide & conquer approach.

Algorithm

1. **Divide:** keep dividing array into two halves until it cannot be divided.
2. **Conquer:** Each subarray is sorted individually using merge sort algorithm.
3. **Combine:** The sorted subarrays are merged back together in sorted order. This process continues until all the elements from both subarrays have been merged.

Time Complexity

- Best Case: when the array is already sorted or nearly sorted. $O(n \log n)$
- Average Case: when the array is randomly ordered. $O(n \log n)$
- Worst Case: when the array is sorted in reverse order. $O(n \log n)$

Application of Merge Sort

- Sorting large dataset
- External sorting, when dataset is too large to fit in memory.

Advantages of Merge Sort

- Merge Sort is a stable sorting algorithm.
- In worst case merge sort has time complexity $O(n \log n)$
- We can perform merging parallelly. Means this problem is suitable for parallel processing system.

Disadvantage of Merge Sort

- Merge sort require extra memory to store the merged sub-arrays during the sorting process.

Quick Sort

- Quick Sort is based on Divide & Conquer

Algorithm

1. **Pick a Pivot:** Choose an element from the array as the pivot. The choice of pivot can be first element, last element, median element or random element.
2. **Partition the Array:** Pivot is placed on its right place and all elements smaller than the pivot will be on its left and all elements greater than the pivot on its right.
3. **Recursively Call:** Recursively apply the step 1 and step 2 to the two partitioned subarray.

Time Complexity

- **Best Case:** When the pivot element divides the array into two equal halves.

Big Omega($n \log n$)

- **Average Case:** The pivot divides array into two halves but not necessarily equal.

Theta($n \log n$)

- **Worst Case:** When smallest or largest element is always chosen as pivot in sorted array.

$O(n^2)$

Space Complexity

- Worst Case: Unbalanced partition leading to a skewed recursion tree which makes call stack size $O(n)$.
- Best Case: Balanced partitioning leading to balanced recursion tree which makes call stack size $O(\log n)$.

Note

- Quick Sort is not a stable sorting algorithm